Informatica® Cloud Application Integration
August 2025

OpenAI Connector Guide

Informatica Cloud Application Integration OpenAI Connector Guide
August 2025

© Copyright Informatica LLC 2024, 2025

Publication Date: 2025-08-30

Table of Contents

Preface 4

Chapter 1: OpenAI Connector overview 5

Introduction to OpenAI 5

Chapter 2: OpenAI connections overview 6

Basic connection properties 6

OpenAI connection properties 7

OpenAI connection metadata 7

OpenAI connection actions and objects 8

Using an OpenAI connection in a process 12

Rules and guidelines for using OpenAI connector 13

Preface

Read the *OpenAI Connector Guide* to learn how to set up and use OpenAI connections. This guide assumes that you have an understanding of the OpenAI API concepts.

CHAPTER 1

OpenAI Connector overview

OpenAI Connector is a suite of tools and integrations designed to help you connect OpenAI's powerful language models and AI technologies with a variety of applications, platforms, and services. It streamlines the process of integrating OpenAI's models into different environments, making it easy for developers and businesses to leverage AI capabilities.

OpenAI Connector connects to OpenAI models through APIs. OpenAI includes APIs for models such as GPT-4, Codex, and DALL·E. OpenAI Connector also handles API key management and usage limits to ensure secure and efficient interactions with OpenAI's services.

Use OpenAI Connector to integrate with third-party tools that offer pre-built connectors for popular platforms and tools, making it easy to integrate OpenAI models without extensive custom development. You can integrate with CRM systems such as Salesforce to automate customer interactions and data management. You can also integrate with platform connectors such as Slack or Microsoft Teams to integrate AI capabilities into communication and collaboration tools.

You can use OpenAI Connector to achieve the following general use cases:

- Integrate chatbots and virtual assistants into customer service platforms to handle inquiries
- Provide information, code assistance, and manage customer interactions
- Content creation, creative applications, and educational tools
- Research and analysis
- Data analysis to gain insights

Introduction to OpenAI

The OpenAI API provides a simple interface for developers to create an intelligence layer in their applications. The Chat Completions endpoint powers ChatGPT and provides a simple way to take text as the input and use a model like GPT-4o to generate the output.

CHAPTER 2

OpenAI connections overview

Use an OpenAI connection for chat completions, file embeddings, and batch requests using an API key.

You can perform the following actions using the OpenAI API:

- Create embeddings and chat completion
- Retrieve a model instance with the basic information or the list of models available in the API
- Upload or delete files
- Create batch, retrieve file or file content, separately or in batches

After you create an OpenAI connection, you can validate, test, and save the connection.

You can then publish the OpenAI connection and click the **Metadata** tab to view the generated process objects for the connection.

Basic connection properties

The following table describes the basic properties that you can configure on the **Properties** tab of the connection creation page:

Property	Description
Name	Required. Unique name for the OpenAI connection that identifies it in the Process Designer. The name must start with an alphabet and can contain only alphabets, numbers, and hyphens (-).
Location	The location of the project or folder where you want to save the connection. Click **Browse** to select a location. If the **Explore** page is currently active and a project or folder is selected, the default location for the connection is the selected project or folder. Otherwise, the default location is the location of the most recently saved asset.
Description	Optional. Description of the connection.
Type	Required. The type of connection you want to use for the connector or service connector. Select **OpenAI**.
Runtime Environment	Required. The runtime environment for the connection. You can run the connection on the Cloud Server, a Secure Agent group, or a Secure Agent machine.

Property	Description
Connection Test	Not supported for OpenAI Connector.
OData-Enabled	Not supported for OpenAI Connector.

OpenAI connection properties

To use the OpenAI API, you need an API key. Enter the API key in the **API_Key** property in the **Connection Properties** section. The OpenAI API uses the API key to authenticate OpenAI connection requests.

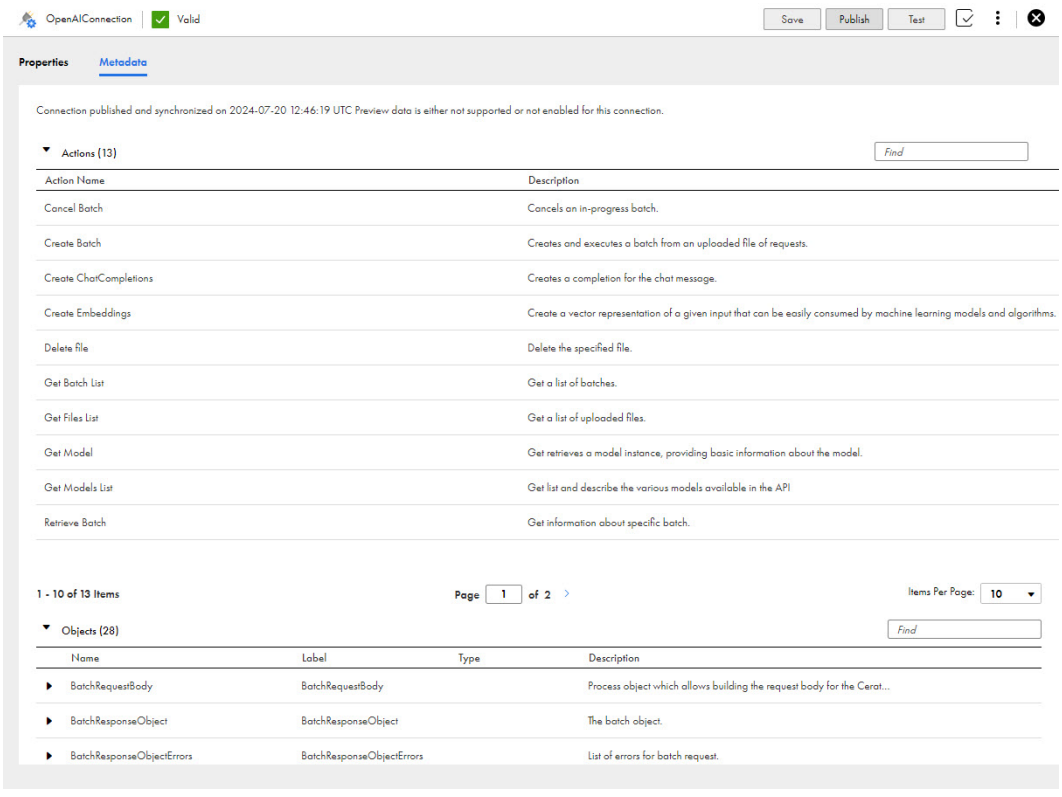
For information about creating an API key, see the OpenAI API documentation.

OpenAI connection metadata

After you create an OpenAI connection, you can validate and save the connection. You can then publish the OpenAI connection and click the **Metadata** tab to view the generated process objects for the connection.

When you publish an OpenAI connection in Application Integration, a list of **Actions** and **Objects** appears on the **Metadata** tab.

The following image shows the **Metadata** tab for a published OpenAI connection:

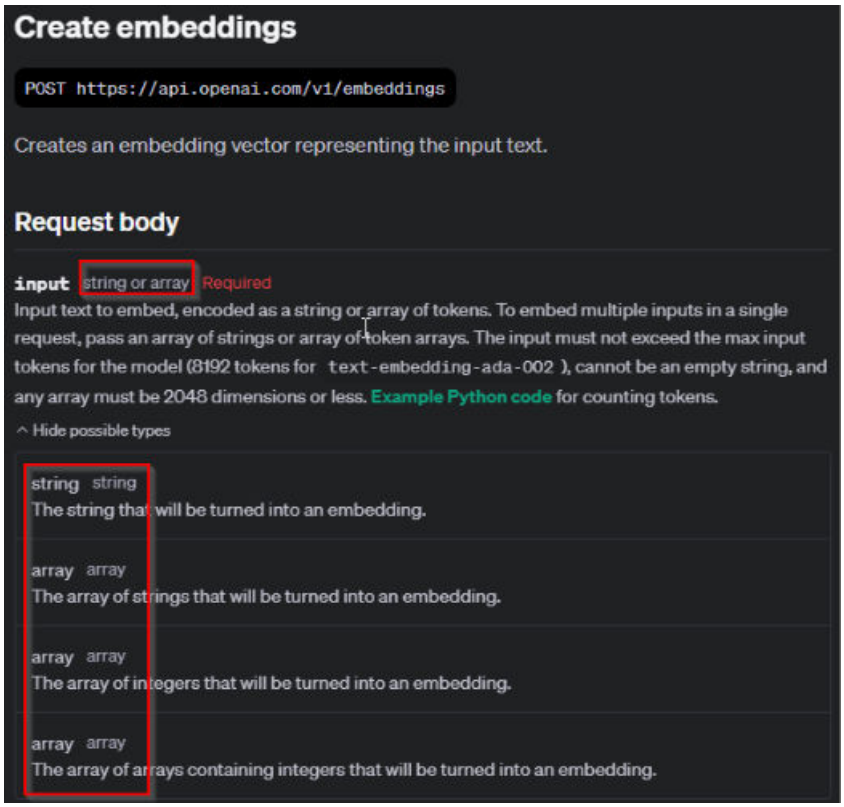


OpenAI connection actions and objects

When you publish an OpenAI connection in Application Integration, the **Actions** and **Objects** appear on the **Metadata** tab.

Consider the following information when you use specific actions and objects:

- The **Create Embeddings** action creates a vector representation of an input that machine learning algorithms can easily consume.
  You can create a request body with an input field with different data types, such as string or array as shown in the following image:



  If the input field must be in string format, you do not need to add additional parameters, as shown in the following request body snippet:

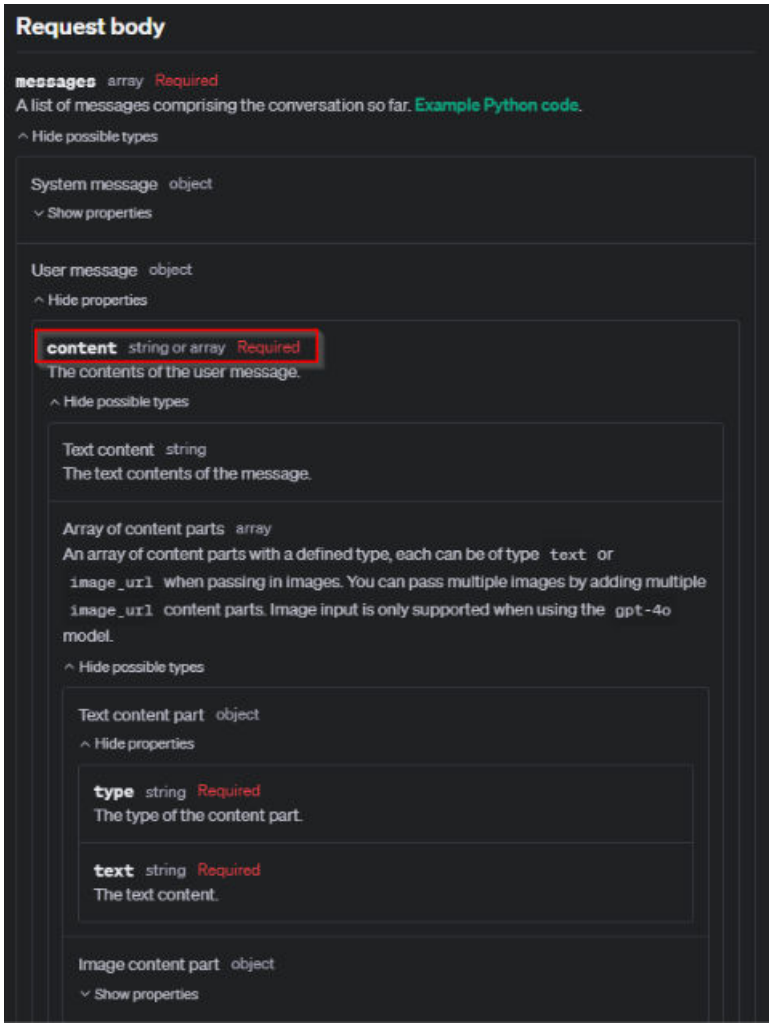
```
<root>
    <input>The food was delicious and the waiter...</input>
    <model>text-embedding-3-large</model>
    <encoding_format>float</encoding_format>
    <dimensions>10</dimensions>
    <user>userName</user>
</root>
```

  However, if the input field must be in array format, you need to include additional namespaces as shown in the following request body snippet:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <input m:isArray="true">The food was delicious and the waiter...</input>
    <model>text-embedding-3-large</model>
    <encoding_format>float</encoding_format>
    <dimensions>10</dimensions>
    <user>userName</user>
</root>
```

- The **Create ChatCompletions** action creates a completion for the chat message.

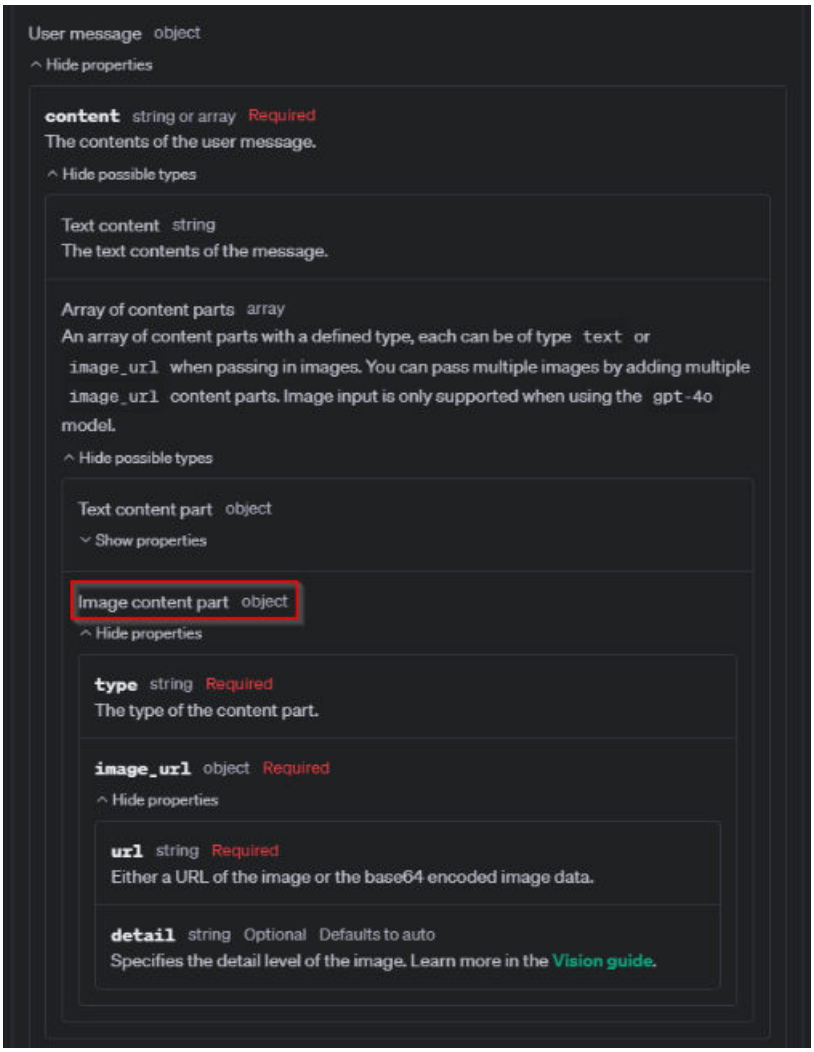
You can create a request body with content, tool_choice, and other fields with different data types, such as `string or array` or `string or object`, as shown in the following image:



The following snippet is a sample request body with content fields of different types. You do not need to add additional namespaces in the request body.

```
<root>
    <model>gpt-4o</model>
    <messages>
        <role>system</role>
        <content>This is string content field</content>
    </messages>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>This is array content field</text>
        </content>
    </messages>
</root>
```

You can transfer an image to OpenAI using one of the following options:



- Use the image link as shown in the following request body example:

```
<root>
    <model>gpt-4-turbo</model>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>What's in this image?</text>
        </content>
        <content>
            <type>image_url</type>
            <image_url>
            <url>https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg</url>
            </image_url>
        </content>
    </messages>
    <max_tokens>300</max_tokens>
</root>
```

- Use base64 encoded image data as shown in the following request body example:

```
<root>
    <model>gpt-4-turbo</model>
```

```
        <messages>
            <role>user</role>
            <content>
                <type>text</type>
                <text>What's in this image?</text>
            </content>
            <content>
                <type>image_url</type>
                <image_url>
                    <url>[base64 encode here]</url>
                </image_url>
            </content>
        </messages>
        <max_tokens>300</max_tokens>
    </root>
```

- The **Get Models List** action gets a list that describes the various models available in the API.
- The **Get Model** action retrieves a model instance that provides basic information about the model.
- The **Upload File** action uploads a file that can be used with batch actions.
- The **Get Files List** action gets a list of uploaded files.
- The **Retrieve File** action gets information about a specific file.
- The **Delete file** action deletes the specified file.
- The **Retrieve File Content** action gets the content of the specified file.
- The **Create Batch** action creates and executes a batch from an uploaded file of requests.
- The **Retrieve Batch** action gets information about a specific batch.
- The **Cancel Batch** action cancels an in-progress batch.
- The **Get Batch List** action gets a list of batches.
- The **Create Model Response** action creates a single model response.
  The following snippet is a sample request body:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <model>gpt-4o-mini</model>
    <input>Tell me a three sentence bedtime story about a frendship between Messi and
Ronaldo.</input>
    <background m:type="xs:boolean">true</background>
</root>
```

  You can also use the MCP server with parameters as shown in the following snippet:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <model>gpt-4o-mini</model>
    <input>Add the Blemish Toner Pads to my cart</input>
    <temperature m:type="xs:double">0.2</temperature>
    <max_output_tokens m:type="xs:int">300</max_output_tokens>
    <tools m:isArray="true">
        <type>mcp</type>
        <server_label>shopify</server_label>
        <server_url>https://your-mcp-host.com/mcp</server_url>
        <headers>
            <Authorization>Bearer YOUR_MCP_SERVER_KEY</Authorization>
        </headers>
        <allowed_tools m:isArray="true">add_to_cart</allowed_tools>
        <allowed_tools m:isArray="true">checkout</allowed_tools>
    </tools>
</root>
```

  Create Model Response action returns the unique identifier. For example, `resp_67ccd2bed1ec8190b14f964abc0542670bb6a6b452d3795b`.
- The **Get Model Response** action accepts the model response ID as input that is generated by the Create Model Response action and returns a single model response.

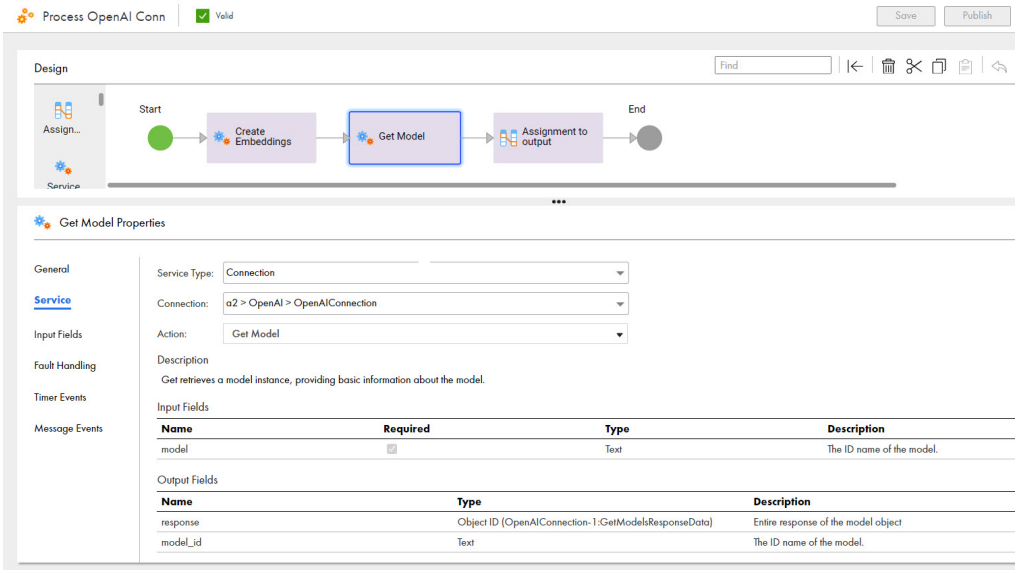
- The **Delete Model Response** action accepts the model response ID as input that is generated by the Create Model Response action, and deletes a model response and returns the deleted status.
- The **Cancel Model Response** action accepts the model response ID as input that isgenerated by the Create Model Response action and revokes it.
- The **Get List Items of Response** action returns a list of items from the generated model response. It accepts response ID that is generated by the Create Model Response action.
  For other types, use parameters as shown in the following snippet:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

Using an OpenAI connection in a process

After you create an OpenAI connection, you can use it in a Service step in a process.

1. Add a Service step to the process.
2. Click the **Service** tab.
3. From the **Service Type** list, select **Connection**.
4. From the **Connection** list, browse and select the OpenAI connection that you created.
5. From the **Action** list, select the action that you want to perform on OpenAI Connector. For example, select **Get Model** to retrieve a model instance that provides basic information about the model as shown in the following image:



   In this case, Application Integration populates the **response** and **model_id** object under the output fields. Similar to the Get Model step, you can configure more service steps as required.
6. Configure further steps as needed.
7. Validate, save, and publish the process.

Rules and guidelines for using OpenAI connector

Consider the following rules and guidelines when using the OpenAI connector:

- The Chat completions action enables you to create, define, and use functions as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <model>gpt-4-turbo</model>
    <messages>
        <role>user</role>
        <content>What's the weather like in Boston today?</content>
    </messages>
    <tools>
        <type>function</type>
        <function>
            <name>get_current_weather</name>
            <description>Get the current weather in a given location</description>
            <parameters>
                <type>object</type>
                <properties>
                    <location>
                        <type>string</type>
                        <description>The city and state, e.g. San Francisco, CA</
description>
                    </location>
                    <unit>
                        <type>string</type>
                        <enum>celsius</enum>
                        <enum>fahrenheit</enum>
                    </unit>
                </properties>
                <required m:isArray="true">location</required>
            </parameters>
        </function>
    </tools>
    <tool_choice>auto</tool_choice>
</root>
```

- You can define the Embeddings input property with multiple data types, such as string or array. If the input is an array, you must add array type annotation as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <input m:isArray="true">The food was delicious and the waiter...</input>
    <model>text-embedding-3-large</model>
    <encoding_format>float</encoding_format>
    <dimensions>10</dimensions>
    <user>userName</user>
</root>
```

- You can define the Chat Completions content property with multiple data types, such as string or array. If the content is an array, you must not use additional spaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <model>gpt-4o</model>
    <messages>
        <role>system</role>
        <content>This is string content field</content>
    </messages>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>This is array content field</text>
        </content>
    </messages>
</root>
```

- You can use an image in the Chat Completion action as shown in the following sample code:

```
<root>
    <model>gpt-4-turbo</model>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>What's in this image?</text>
        </content>
        <content>
            <type>image_url</type>
            <image_url>
            <url>https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-
wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-
boardwalk.jpg</url>
            </image_url>
        </content>
    </messages>
    <max_tokens>300</max_tokens>
</root>
```

- To use the **logit_bias** field in the **map** type key-value pairs, you must use a special requestBody structure for the requests as shown in the following sample code:

```
<root>
    ...
    <logit_bias>
        <item>
            <id>5935</id>
            <bias>-100</bias>
        </item>
        <item>
            <id>9653</id>
            <bias>-100</bias>
        </item>
        <item>
            <id>1130</id>
            <bias>-100</bias>
        </item>
    </logit_bias>
</root>
```

- To create a chat, you must add previous messages and responses to the requestBody. By doing this, you can accumulate the entire chat history. You can create a chat as shown in the following sample code:

```
<root>
    <model>gpt-4o</model>
    <messages>
        <role>system</role>
        <content>
            <type>text</type>
            <text>You are a helpful assistant. You should answer briefly</text>
        </content>
    </messages>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>What is a LLM?</text>
        </content>
    </messages>
    <messages>
        <role>assistant</role>
        <content>
            <type>text</type>
            <text>A Language Model (LLM) is an artificial intelligence model designed
to understand, generate, and manipulate human language. It is trained on vast amounts
of text data to perform tasks like translation, summarization, and conversation.
Examples include OpenAI's GPT-3 and GPT-4.</text>
        </content>
```

```
    </messages>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>Okay. What is GPT?</text>
        </content>
    </messages>
    <messages>
        <role>assistant</role>
        <content>
            <type>text</type>
            <text>GPT stands for Generative Pre-trained Transformer. It's a type of
language model developed by OpenAI that can generate human-like text based on the
input it receives. GPT models are pre-trained on extensive datasets and can perform
various tasks such as translation, summarization, and conversational AI.</text>
        </content>
    </messages>
    <messages>
        <role>user</role>
        <content>
            <type>text</type>
            <text>What is the difference between LLM and GPT?</text>
        </content>
    </messages>
</root>
```

- To use fields with data types, such as array, boolean, double, or integer, you must use annotation as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">10.3</field_double>
    <field_integer m:type="xs:int">5</field_integer>
</root>
```